

Messaging Protocol Document

IoT Measurement Setup

M. Rats, V. Gülmez, G. Gharib, C. Stijlaart

Table of Contents

Introduction..... 1

Description..... 1

Message Format 1

Messages..... 3

Introduction

This document outlines the messaging protocol designed for an IoT Measurement Setup. The protocol facilitates the exchange of commands, data, and status updates, enabling seamless coordination and control of IoT devices involved in measurement processes.

Description

The IoT Measurement Setup involves the interaction between distinct units such as Sensor Units, Control Units, and Camera Units. These units communicate through a well-defined messaging format, allowing communication with specific parts within the system.

Message Format

The message format created by using our Send() function in the MessageHandler (class). Each argument in the Send() function presents the corresponding <value>.

```
{
  "destination" : "<value>",
  "source" : "<value>",
  "command" : "<valie>",
  "data" : {

  },
  "timestamp" : "<value>"
}
```

Request example:

This message instructs the "SensorUnit" to provide data related to the sensor with the identifier "temperature_1." The request originates from the "ControlUnit."

```
{
  "destination" : "SensorUnit",
  "source" : "ControlUnit",
  "command" : "GetSensorData",
  "data" : {
    "sensor_id" : "temperature_1"
  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Response example:

This message transmits data to the "ControlUnit" Providing data related to the sensor with the identifier "temperature_1." The transmission originates from the "SensorUnit."

```
{
  "destination" : "ControlUnit",
  "source" : "SensorUnit",
  "command" : "GetSensorData",
  "data" : {
    "sensor_id" : "temperature_1",
    "value" : "13.7"
  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Request example:

When the camera (CameraUnit) must be started by sending via the "ControlUnit" the command "Start".

```
{
  "destination" : "CameraUnit",
  "source" : "ControlUnit",
  "command" : "Start",
  "data" : {

  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Response example:

When the "CameraUnit" has started and sends back "OK" to the "ControlUnit" letting it know it started. This could send "error" back instead, indicating that it couldn't start.

```
{
  "destination" : "ControlUnit",
  "source" : "CameraUnit",
  "command" : "Start",
  "data" : {
    "return" : "OK"
  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Request example:

When the camera (CameraUnit) must be stopped by sending via the "ControlUnit" the command "Stop".

```
{
```

```
"destination" : "CameraUnit",
"source" : "ControlUnit",
"command" : "Stop",
"data" : {

},
"timestamp" : "11:12:30 19-12-2023"
}
```

Response example:

When the "CameraUnit" has stopped and sends back "OK" to the "ControlUnit" letting it know it stopped.

```
{
  "destination" : "ControlUnit",
  "source" : "CameraUnit",
  "command" : "Stop",
  "data" : {
    "return" : "OK"
  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Camera event example:

When the camera (CameraUnit) event is triggered for e.g. detecting a color change. "CameraUnit" is telling the "ControlUnit" an event occurred. In the data field there is a subfield called "Timestamp" with a given time "11:12:29" indicating the time of the event happening and a subfield called "EventType" indicating the event is a color change.

```
{
  "destination" : "ControlUnit",
  "source" : "CameraUnit",
  "command" : "EventOccured",
  "data" : { "EventType":"ColorChange",
    "Timestamp":"11:12:29"
  },
  "timestamp" : "11:12:30 19-12-2023"
}
```

Messages

message	paramaters	description	source	destination
getData	string collection_date	retrieves data from the database	*	control unit (database manager)
addData	string data (json)	adds the given data to the database, to the signed in user.	*	control unit (database manager)
removeCollection	string collection_date	deletes the all the data from te given collection for the signed in user.	*	control unit (database manager)
signOut	-	logs out of the current user, and goes back to the guest user.	*	control unit (database manager)
signIn	string username, string password	signs in into the given user.	*	control unit (database manager)
currentUser	-	retrieves the current active user.	*	control unit (database manager)
removeUser	-	removes the given user from the access list of the entire database	*	control unit (database manager)
addUser	string username, string password	adds a user to the database	*	control unit (database manager)
giveAccess	string access_group	gives the given group access to the database of the signed in user.	*	control unit (database manager)
removeAccess	string acces_group	removes the access from the given group to the database of the current signed in user	*	control unit (database manager)

message	paramaters	description	source	destination
Start	-	starts the experiment	control unit	control unit (video manager), camera unit (camera manager), sensor unit (sensor manager)
Stop	-	stops the experiment	control unit	control unit (video manager), camera unit (camera manager), sensor unit (sensor manager)
EventOccured	string EventType, string timestamp	returns what type of event has occured and when	camera unit	control unit (database manager)
GetSensorData	string sensor_id	retrieves the sensor data of the given sensor	control unit	sensor unit (sensor manager)